1 Introduction

Branch Predictor Implementation
Author: **Ahmed Ahmed**
October 25, 2026

GitHub repository:
https://github.com/ahmedshehata08-iss/branch-predictor

In this assignment, I completed the `pm_predictor` class in the supplied CBP2 infrastructure. I combined a bimodal predictor with a tagged, two-way global predictor so that a global hit supplies the direction prediction and a global miss uses the bimodal result.

The original class predicted every branch as taken and did not update any state. I added the tables, two-bit counters, global history, tag matching, allocation, and least-recently-used replacement. I also added repeatable tests and measurements for all 20 supplied traces.

What works

• A 4,096-entry bimodal predictor and a 512-set, two-way global predictor.

• Valid/tag checks, global-hit priority, saturating counters, and invalid-first/LRU replacement.

• Actual-outcome history updates and isolation of unconditional branches.

• Original single-trace use, a portable suite runner, exact statistics, and four predictor comparisons.

Measured outcome

The completed hybrid averaged **7.260 MPKI** and achieved **94.73%** pooled conditional-branch accuracy. Mean MPKI fell by **91.59%** relative to the original always-taken behavior and by **29.51%** relative to bimodal alone. The supplied, larger gshare predictor averaged 6.306 MPKI and was better overall.

Scope

I implemented the required direction-prediction subset. I did not implement the optional loop predictor, PIR, branch targets, BTB, indirect target cache, or complete extra-credit predictor. This is an educational Pentium M-style model, not a claim of exact commercial-hardware behavior.

The report follows the supplied sample: introduction, code changes labeled **Initial Version** and **My Version**, then tests and results. Every code image is rendered from the actual packaged source with its original line numbers.

2 Predictor design and assumptions

Component	Default organization / policy
Bimodal	4,096 counters; two logical bits each; initialized to 1 (weakly not taken).
Global	512 sets x 2 ways; each entry has a valid bit, 6-bit tag, and two-bit counter.
History	15 actual conditional outcomes; initial value 0.
Replacement	Prefer an invalid way; otherwise evict LRU. One victim bit per set.
Counter decision	0 or 1 predicts not taken; 2 or 3 predicts taken.
New global entry	Allocate on every global miss; initialize to 2 for T or 1 for NT.
Training	Always train bimodal for conditional branches; train the hit entry or allocate on a miss.

Indexing and prediction

```
bimodal_index = PC & 0xFFF
hash = (PC ^ GHR) & 0x7FFF
row = hash >> 6 # HASH[14:6]
tag = hash & 0x3F # HASH[5:0]
final = global[row][hit_way] if valid_tag_hit else bimodal[index]
```

I use raw PC bits in the default model, matching the main assignment diagram. The Appendix B teaching example shifts the PC by two bits; I expose that as a separate small configuration rather than mixing its address convention into the default benchmark model.

Update order and storage

The driver predicts, scores the prediction, and then updates using the actual outcome. I retain the lookup fields in `pm_update`, train the counters, refresh LRU, and finally shift the outcome into GHR. Nonconditional branches do not change this state.

Logical predictor state is 4,096 x 2 + 512 x 2 x (1 + 6 + 2) + 512 + 15 = **17,935 bits** (about 2.19 KiB). This excludes simulator bookkeeping. The C++ representation uses bytes, integers, vectors, and padding, so its host memory usage is larger.

3.1 Saved lookup fields and table state

I expanded the update record to preserve the exact lookup used during prediction. I also introduced global entries, table storage, and an LRU victim array. The original class had no predictor tables or history.

Initial Version

baseline/my_predictor.h | lines 57-67

```
57  class pm_update : public branch_update {
58  public:
59          unsigned int index;
60  };
61
62  class pm_predictor : public branch_predictor {
63  public:
64          pm_update u;
65
66          pm_predictor (void) {
67          }
```

My Version

src/my_predictor.h | lines 61-88

```
61  // Prediction-time lookup information; the driver resolves one branch at a time.
62  class pm_update : public branch_update {
63  public:
64      bool conditional = false;
65      unsigned int index = 0;   // bimodal index
66      unsigned int row = 0;
67      unsigned int tag = 0;
68      int way = -1;             // -1 means the global lookup missed
69  };
70
71  // A simplified Pentium M-style hybrid direction predictor.
72  // Default mapping follows the assignment diagram: raw IP bits, not IP >> 2.
73  class pm_predictor : public branch_predictor {
74  public:
75      struct global_entry {
76          bool valid = false;
77          unsigned int tag = 0;
78          unsigned char counter = 1;
79      };
80      struct global_set { global_entry ways[2]; };
81
82      pm_update u;
83      unsigned int history = 0;
84      unsigned int bimodal_mask, row_mask, tag_mask, history_mask;
85      unsigned int tag_bits, pc_shift;
86      std::vector<unsigned char> bimodal;
87      std::vector<global_set> global;
88      std::vector<unsigned char> lru; // next victim if both ways are valid
```

Figure 1. Original state declaration and the new prediction context/table declarations.

A global hit requires both `valid` and a matching tag. The `way = -1` value marks a miss and selects the bimodal fallback. Keeping row, tag, and index in the update object prevents training a different entry after history changes.

3.2 Initialization and saturating counters

I replaced the empty constructor with deterministic initialization. All bimodal counters start weakly not taken; global entries start invalid. Constructor parameters allow small test configurations while the no-argument constructor retains the assignment sizes.

Initial Version

baseline/my_predictor.h | lines 66-67

```
66  pm_predictor (void) {
67  }
```

My Version

src/my_predictor.h | lines 90-112

```
90  explicit pm_predictor(unsigned int bimodal_bits = 12,
91                        unsigned int set_bits = 9,
92                        unsigned int tag_bits_arg = 6,
93                        unsigned int pc_shift_arg = 0) {
94      if (bimodal_bits < 1 || bimodal_bits > 16 ||
95          set_bits < 1 || set_bits > 16 ||
96          tag_bits_arg < 1 || tag_bits_arg > 8 || pc_shift_arg > 31)
97          throw std::invalid_argument("invalid predictor configuration");
98      tag_bits = tag_bits_arg;
99      pc_shift = pc_shift_arg;
100     bimodal_mask = (1u << bimodal_bits) - 1;
101     row_mask = (1u << set_bits) - 1;
102     tag_mask = (1u << tag_bits) - 1;
103     history_mask = (1u << (set_bits + tag_bits)) - 1;
104     bimodal.assign(1u << bimodal_bits, 1); // weakly not taken
105     global.resize(1u << set_bits);       // all entries invalid
106     lru.assign(1u << set_bits, 0);
107 }
108
109 static void train(unsigned char &counter, bool taken) {
110     if (taken && counter < 3) ++counter;
111     if (!taken && counter > 0) --counter;
112 }
```

Figure 2. Constructor and shared two-bit training helper.

Current counter	After NT	After T	Prediction before update
0	0	1	NT
1	0	2	NT
2	1	3	T
3	2	3	T

The counter saturates at 0 and 3. On allocation I set a fresh weak state for the observed outcome; I do not inherit a replacement victim's old bias. The assignment leaves this policy open, so I document it explicitly.

3.3 Global lookup and final prediction

I replaced unconditional taken prediction with the required hybrid selection. Both global ways are checked. A matching valid tag wins even when the bimodal counter disagrees; otherwise the bimodal prediction is returned.

Initial Version

baseline/my_predictor.h | lines 69-78

```
69          branch_update *predict (branch_info & b) {
70
71              // predict branch outcome
72              u.direction_prediction (true);
73
74              // predict branch target address
75              u.target_prediction (0);
76
77              return &u;
78          }
```

My Version

src/my_predictor.h | lines 114-138

```
114 branch_update *predict(branch_info &b) override {
115     u = pm_update();
116     u.conditional = (b.br_flags & BR_CONDITIONAL) != 0;
117     u.target_prediction(0); // target prediction is outside the assignment
118     if (!u.conditional) {
119         u.direction_prediction(true);
120         return &u;
121     }
122     const unsigned int pc = b.address >> pc_shift;
123     u.index = pc & bimodal_mask;
124     const unsigned int hash = (pc ^ history) & history_mask;
125     u.row = (hash >> tag_bits) & row_mask;
126     u.tag = hash & tag_mask;
127     for (int way = 0; way < 2; ++way) {
128         const global_entry &entry = global[u.row].ways[way];
129         if (entry.valid && entry.tag == u.tag) {
130             u.way = way;
131             break;
132         }
133     }
134     const unsigned char counter = (u.way >= 0)
135         ? global[u.row].ways[u.way].counter : bimodal[u.index];
136     u.direction_prediction(counter >= 2);
137     return &u;
138 }
```

Figure 3. Prediction before and after implementation. The target remains zero by design.

Prediction reads the tables without training them. Unconditional branches return taken immediately. The global hash uses the history that existed before the current branch outcome became known.

3.4 Training, allocation, LRU, and history

I implemented the previously empty update function. Every conditional outcome trains the indexed bimodal counter. A global hit trains the matched entry; a miss allocates an invalid entry or replaces the least recently used way.

Initial Version

baseline/my_predictor.h | lines 80-82

```
80  void update (branch_update *u, bool taken, unsigned int target) {
81
82  }
```

My Version

src/my_predictor.h | lines 140-161

```
140 void update(branch_update *result, bool taken,
141             unsigned int /*target*/) override {
142     const pm_update &lookup = *static_cast<pm_update *>(result);
143     if (!lookup.conditional) return;
144     train(bimodal[lookup.index], taken);
145     int way = lookup.way;
146     if (way < 0) {
147         // Fill an invalid way first; otherwise evict the least recent way.
148         if (!global[lookup.row].ways[0].valid) way = 0;
149         else if (!global[lookup.row].ways[1].valid) way = 1;
150         else way = lru[lookup.row];
151         global_entry &entry = global[lookup.row].ways[way];
152         entry.valid = true;
153         entry.tag = lookup.tag;
154         entry.counter = taken ? 2 : 1; // fresh weak state, no stale bias
155     } else {
156         train(global[lookup.row].ways[way].counter, taken);
157     }
158     lru[lookup.row] = static_cast<unsigned char>(1 - way);
159     history = ((history << 1) | static_cast<unsigned int>(taken))
160               & history_mask;
161 }
```

Figure 4. The complete hybrid update path.

For two ways, one victim bit is sufficient for exact LRU: after using way 0, way 1 becomes the victim; after using way 1, way 0 becomes the victim. I refresh this state on both hits and allocations.

The last operation shifts the **actual** outcome into the masked history register. The update uses saved lookup indices, not a newly calculated hash. The serial driver resolves each branch immediately, so only one prediction context is outstanding.

3.5 Measurement and comparison controls

I kept direction scoring before update and added exact conditional counts, accuracy, and separate global-hit/fallback miss counts. The driver accepts `pm`, `bimodal`, `gshare`, or `always_taken`; its default remains the required hybrid.

Initial Version

baseline/predict.cc | lines 58-67

```
58  if (t->bi.br_flags & BR_CONDITIONAL) {
59
60      // count a direction misprediction
61
62      dmiss += u->direction_prediction () != t->taken;
63
64      // count a target misprediction
65
66      tmiss += u->target_prediction () != t->target;
67  }
```

My Version

src/predict.cc | lines 33-53

```
33  while (trace *t = read_trace()) {
34      branch_update *result = predictor->predict(t->bi);
35      ++branches;
36      if (t->bi.br_flags & BR_CONDITIONAL) {
37          ++conditional;
38          const bool wrong = result->direction_prediction() != t->taken;
39          misses += wrong;
40          if (is_pm) {
41              const bool hit = static_cast<pm_update *>(result)->way >= 0;
42              hits += hit;
43              if (hit) hit_misses += wrong;
44              else fallback_misses += wrong;
45          }
46      }
47      predictor->update(result, t->taken, t->target);
48  }
49  end_trace();
50  // The supplied CBP2 traces each represent 100 million instructions.
51  const double mpki = 1000.0 * static_cast<double>(misses) / 100000000.0;
52  const double accuracy = conditional
53      ? 100.0 * (1.0 - static_cast<double>(misses) / conditional) : 0.0;
```

Figure 5. Conditional direction scoring and measured diagnostics.

The existing three-decimal MPKI line is preserved. With `--stats`, a JSON line records exact counts and higher-precision metrics. I removed target-miss reporting from the main experiment because branch targets are outside the required scope.

The always-taken control matches the starter `pm_predictor` behavior. The supplied 32,768-entry gshare algorithm is retained as a reference; the new bimodal-only control uses the same 4,096 counters and initial state as the hybrid fallback.

3.6 Portable run and reproducible build

The original runner required csh and dc. I replaced it with Python 3, preserving sorted trace execution and the average-MPKI result. Commands are passed as argument lists, and a failed child process stops the run.

Initial Version

baseline/run | lines 13-23

```
13 set trace_list = `find $1 -name '*.trace.*' | sort`
14 set sum = 0
15 set n = 0
16 foreach i ( $trace_list )
17     printf "%-40s\t" $i
18     set mpki = `./src/predict $i | tail -1 | sed -e '/MPKI/s///'`
19     printf "%0.3f\n" $mpki
20     set sum = `printf "$sum\n$mpki\n+\np\n" | dc`
21     @ n = $n + 1
22 end
23 printf "average MPKI: "
```

My Version

run | lines 13-20

```
13 for trace in traces:
14     result=subprocess.run([str(root/'src/predict'),str(trace),mode,'--stats'],
15                           check=True,capture_output=True,text=True)
16     import json
17     value=json.loads(result.stdout.splitlines()[0])['mpki']
18     values.append(value)
19     print(f'{str(trace):55s} {value:.3f}',flush=True)
20 print(f'average MPKI: {sum(values)/len(values):.3f}')
```

Figure 6. Original csh aggregation and the Python runner's corresponding loop.

I updated the Makefile to compile C++17 with warnings and added separate `test` and `sanitize` targets. A dedicated comparison script records all 80 runs, their exact outputs, and a CSV result table.

The main build produces only the required `predict` program. The inherited `cpm_predictor` remains an unimplemented optional stub, available only through the explicitly named `extra-credit` target. This prevents it being mistaken for a completed submission.

3.7 Trace opening and error handling

While validating the command line, I found that the original trace reader continued after a failed file open. I added an immediate error exit, checked reads of the compression signature, and used a const decompressor string. The trace-decoding algorithm remains unchanged.

Initial Version

baseline/trace.cc | lines 426-435

```
426  // figure out the compression method from the magic number
427
428  FILE *f = fopen (fname, "r");
429  if (!f) {
430      perror (fname);
431  }
432  fread (s, 1, 2, f);
433  fclose (f);
434  if (strncmp (s, GZIP_MAGIC, 2) == 0)
435      dc = ZCAT;
```

My Version

src/trace.cc | lines 429-440

```
429  FILE *f = fopen (fname, "rb");
430  if (!f) {
431      perror (fname);
432      exit (1);
433  }
434  const size_t magic_size = fread (s, 1, 2, f);
435  if (ferror (f)) { perror (fname); fclose (f); exit (1); }
436  fclose (f);
437  if (magic_size == 2 && strncmp (s, GZIP_MAGIC, 2) == 0)
438      dc = ZCAT;
439  else if (magic_size == 2 && strncmp (s, BZIP2_MAGIC, 2) == 0)
440      dc = BZCAT;
```

Figure 7. File-open and compression-signature handling.

I also replaced `fclose(tracefp)` with `pclose(tracefp)` and check its return status. A failed gzip/bzip2 process now makes the program fail instead of printing a misleading successful MPKI result.

The CLI tests include a missing filename and a broken bzip2 stream. These checks cover the entry points changed here; the inherited decoder is still intended for valid supplied CBP2 data and is not a complete malformed-record validator.

3.8 Trace filenames with spaces or quotes

The original reader inserted the filename directly into a fixed-size shell command. I changed it to a dynamically sized string and quote the filename as one shell argument, including embedded apostrophes. This makes ordinary paths with spaces work reliably.

Initial Version

baseline/trace.cc | lines 439-447

```
439     dc = CAT;
440
441 // make a command that will decompress the file to stdout
442
443 sprintf (cmd, "%s %s", dc, fname);
444
445 // pipe that stdout to tracefp
446
447 tracefp = popen (cmd, "r");
```

My Version

src/trace.cc | lines 446-460

```
446 // Quote a filename as one shell argument (including embedded apostrophes).
447 std::string quoted = "'";
448 for (const char *p = fname; *p; ++p) {
449     if (*p == '\'') quoted += "'\\''";
450     else quoted += *p;
451 }
452 quoted += "'";
453 const std::string cmd = std::string(dc) + " " + quoted;
454
455 // pipe that stdout to tracefp
456
457 tracefp = popen (cmd.c_str(), "r");
458 if (!tracefp) {
459     perror (fname);
460     exit (1);
```

Figure 8. Decompressor command construction and launch.

The integration test creates a compressed trace whose filename contains both spaces and an apostrophe, runs the actual driver, and checks the branch counts and accuracy. It also checks that the original single-trace command still prints one MPKI line.

The before snapshots throughout this section come from byte-for-byte copies of the supplied files in `baseline/`. The complete change set is available in `results/code_changes.patch`; the images show the relevant excerpts rather than every unchanged line.

4.1 Directed tests and independent reference

I checked the predictor with deterministic cases before interpreting benchmark performance. All **16 directed tests passed**, and the C++ output matched an independent Python dictionary/LRU model for **40,000 events** across four configurations.

Test group	What I verified
Initialization and counters	Table sizes, invalid startup, all 8 saturating transitions.
Addressing	Default hash fields, masked history, saved prediction indices, expected hash aliasing.
Lookup and selection	Invalid matching tags miss; valid wrong tags miss; global hits override bimodal in both directions.
Update and replacement	Both components train on a hit; invalid ways fill first; replacement resets the counter; hits refresh LRU.
History and branch type	Actual outcomes shift once; wrapping is correct; unconditional branches leave direction state unchanged.
Pattern and configuration	Alternating branch pattern is learned; invalid configurations are rejected; first six Appendix B hashes match.

Independent model

The reference test uses per-set ordered dictionaries to maintain recency, rather than reproducing the C++ victim-array representation. A fixed seed (20261025) generates 10,000 events for each of four configurations: (12,9,6,0), (4,2,2,2), (2,1,1,0), and (8,4,3,1). The tuple means bimodal bits, set bits, tag bits, and PC shift.

Each event compares the final prediction, row, tag, hit way, post-update history, selected bimodal counter, and LRU victim. Streams mix conditional and nonconditional branches, small colliding PCs, and full 32-bit addresses.

Sanitizers and command line

AddressSanitizer and UndefinedBehaviorSanitizer passed all directed cases. Leak detection was disabled because the execution container blocks its process inspection; I do not claim a successful leak scan. Four CLI groups check compressed paths/metrics, legacy output, invalid arguments/missing files, and decompression failures.

4.2 Captured test output

This image is an excerpt of the actual successful test run saved in `results/test_output.txt`. The complete log also contains the compiler commands and the sanitizer run.

Captured test output (excerpt)

```
PASS initial_state
PASS all_counter_transitions
PASS hash_and_bit_fields
PASS invalid_tag_is_not_a_hit
PASS tag_mismatch_is_a_miss
PASS hit_overrides_bimodal_both_directions
PASS both_components_train_on_hit
PASS invalid_first_and_fresh_allocation
PASS lru_hit_refresh_and_replacement
PASS actual_history_and_wraparound
PASS unconditional_isolation
PASS saved_prediction_indices
PASS same_hash_aliases
PASS appendix_hash_sequence
PASS alternating_pattern_is_learned
PASS bad_configuration_is_rejected
PASS 16 directed predictor tests
PASS independent reference: (12, 9, 6, 0), 10000 events
PASS independent reference: (4, 2, 2, 2), 10000 events
PASS independent reference: (2, 1, 1, 0), 10000 events
PASS independent reference: (8, 4, 3, 1), 10000 events
PASS 40000 differential events in 4 configurations (seed 20261025)
PASS compressed trace with spaces/apostrophe and exact conditional metrics
PASS original one-trace command and MPKI output format
PASS argument and missing-file errors
PASS decompression failure is not reported as a zero-miss result
```

Figure 9. Captured output from the directed, differential, and command-line tests.

```
make -C src test
make -C src sanitize
```

The final build uses g++ 13.3.0 and Python 3.12.14 in the execution environment. It completed without compiler warnings. The tests and all benchmark commands are included so that the results can be checked on another compatible machine.

4.3 Worked trace using the teaching configuration

I ran the Appendix B branch sequence through the actual C++ predictor with 16 bimodal counters, four sets, two ways, two-bit tags, and PC shifted right by two. Here all bimodal counters start at 1 and allocated global counters start at 2/1, as in my implementation. These reset/allocation rules differ from the appendix's prefilled counters, so the predictions below are my measured results.

#	Br.	Act.	GHR	XOR	Row	Tag	Hit	Pred.	OK
1	b1	T	0000	0001	0	01	N	NT	N
2	b3	T	0001	0101	1	01	N	NT	N
3	b1	NT	0011	0010	0	10	N	T	N
4	b2	NT	0110	1101	3	01	N	NT	Y
5	b3	T	1100	1000	2	00	N	T	Y
6	b1	T	1001	1000	2	00	Y	T	Y
7	b3	T	0011	0111	1	11	N	T	Y
8	b1	NT	0111	0110	1	10	N	T	N
9	b2	NT	1110	0101	1	01	N	NT	Y
10	b3	T	1100	1000	2	00	Y	T	Y
11	b1	NT	1001	1000	2	00	Y	T	N
12	b2	T	0010	1001	2	01	N	NT	N
13	b1	NT	0101	0100	1	00	N	NT	Y
14	b2	NT	1010	0001	0	01	Y	T	N
15	b3	T	0100	0000	0	00	N	T	Y
16	b1	T	1001	1000	2	00	Y	T	Y
17	b3	T	0011	0111	1	11	N	T	Y
18	b1	NT	0111	0110	1	10	N	NT	Y
19	b2	NT	1110	0101	1	01	N	NT	Y
20	b3	T	1100	1000	2	00	Y	T	Y
21	b1	T	1001	1000	2	00	Y	T	Y
22	b3	T	0011	0111	1	11	N	T	Y
23	b1	NT	0111	0110	1	10	N	NT	Y

For example, the first branch has PC 0x44: (0x44 >> 2) & 0xF = 0001. XOR with GHR 0000 gives 0001, hence row 0/tag 01. No valid global tag exists, so the initial bimodal state predicts NT. After actual T, GHR becomes 0001.

5.1 Experimental method and metric definitions

I ran every supplied trace from a fresh predictor state, once for each of four predictors, for **80 completed runs**. There is no warm-up exclusion and no parameter tuning by benchmark. Each trace represents 100 million instructions according to the supplied CBP2 documentation, so the suite represents 2 billion instructions per predictor.

Predictor	Role	Logical counter capacity
Always taken	Original pm stub behavior	No adaptive counter table
Bimodal	Fallback-only control	4,096 x 2 bits = 8,192 bits
Gshare	Supplied reference algorithm	32,768 x 2 bits = 65,536 bits, plus 15 history bits
Hybrid (pm)	Completed assignment	4,096 bimodal + 1,024 global counters; tags/valid/LRU/history included in 17,935 bits

Metrics

```
MPKI = direction_misses / instructions * 1000
Accuracy (%) = (conditional_branches - misses) / conditional_branches * 100
Global hit rate (%) = global_hits / conditional_branches * 100
```

MPKI uses **instructions**, not conditional branch count. Accuracy uses **conditional branches**. All 20 instruction windows are equal length, so the arithmetic mean of per-trace MPKI also equals suite-wide MPKI. Pooled accuracy uses summed misses and summed conditional branches; it is not an unweighted average of percentages.

For synthetic traces I check exact counts and accuracy. The driver's fixed 100-million denominator is valid only for the supplied benchmark windows. Using it on an arbitrary short trace would not give meaningful instruction-normalized MPKI.

Comparison limits

The gshare control has substantially more counter storage. I report it as a supplied reference, not as an equal-budget contest. Its counters begin strongly not taken (0), following the original code; bimodal and hybrid begin weakly not taken (1). The small teaching configuration is used only for tests, not for these benchmark results.

5.2 Complete benchmark MPKI results

Lower MPKI is better. Values below are rounded to three decimals for display; calculations and the packaged CSV retain the measured precision.

Benchmark	Always taken	Bimodal	Gshare	Hybrid (pm)
164.gzip	70.025	15.030	12.473	12.617
175.vpr	85.312	14.671	13.415	14.337
176.gcc	86.071	19.347	11.254	15.298
181.mcf	102.497	33.245	15.837	17.680
186.crafty	48.651	13.075	5.837	8.464
197.parser	78.421	15.417	10.008	10.357
201.compress	76.983	9.356	7.831	8.199
202.jess	92.343	8.809	1.562	1.750
205.raytrace	109.506	3.806	2.756	3.421
209.db	91.451	6.480	3.909	4.613
213.javac	109.644	3.181	2.267	2.638
222.mpegaudio	110.339	3.154	2.188	2.604
227.mtrt	112.022	3.706	2.657	3.268
228.jack	77.958	7.691	3.033	3.946
252.eon	24.226	9.405	1.807	3.139
253.perlbmk	66.833	5.955	2.554	4.700
254.gap	75.950	9.675	3.926	4.797
255.vortex	70.524	1.916	1.222	1.239
256.bzip2	178.257	0.112	0.094	0.076
300.twolf	59.221	21.942	21.489	22.046
Mean	**86.312**	**10.299**	**6.306**	**7.260**

The hybrid improves on bimodal in **19 of 20** traces. The exception is 300.twolf: 22.046 MPKI for hybrid versus 21.942 for bimodal. It beats supplied gshare only on 256.bzip2, which has very few misses under all dynamic predictors.

5.3 Exact hybrid counts and prediction-source use

Benchmark	Conditional	Dir. misses	Accuracy	Global hits
164.gzip	15,114,596	1,261,706	91.652%	83.06%
175.vpr	12,827,221	1,433,739	88.823%	56.09%
176.gcc	17,083,242	1,529,825	91.045%	54.43%
181.mcf	21,700,420	1,767,979	91.853%	77.42%
186.crafty	8,927,032	846,360	90.519%	56.55%
197.parser	14,190,746	1,035,735	92.701%	80.06%
201.compress	11,764,885	819,948	93.031%	87.68%
202.jess	14,154,055	175,020	98.763%	92.36%
205.raytrace	12,201,864	342,149	97.196%	62.95%
209.db	13,191,770	461,313	96.503%	86.70%
213.javac	12,986,593	263,842	97.968%	60.98%
222.mpegaudio	13,021,205	260,376	98.000%	61.23%
227.mtrt	12,512,685	326,838	97.388%	66.69%
228.jack	11,926,271	394,642	96.691%	75.30%
252.eon	7,724,960	313,939	95.936%	76.86%
253.perlbmk	13,537,118	469,974	96.528%	72.93%
254.gap	13,838,041	479,727	96.533%	87.01%
255.vortex	11,073,761	123,933	98.881%	83.95%
256.bzip2	24,586,276	7,602	99.969%	99.62%
300.twolf	13,098,893	2,204,560	83.170%	32.68%

Across the suite, the hybrid makes **14,519,207** direction errors out of **275,461,634** conditional branches. A global hit supplies **203,900,607** predictions (74.02%); the remaining 71,561,027 use the bimodal fallback.

Of the direction errors, 6,739,566 occur on global hits and 7,779,641 on fallback predictions. The corresponding observed error rates are 3.31% and 10.87%. These subsets contain different branch contexts; their rates alone do not establish a causal improvement for every branch.

5.4 Interpretation of the measured results

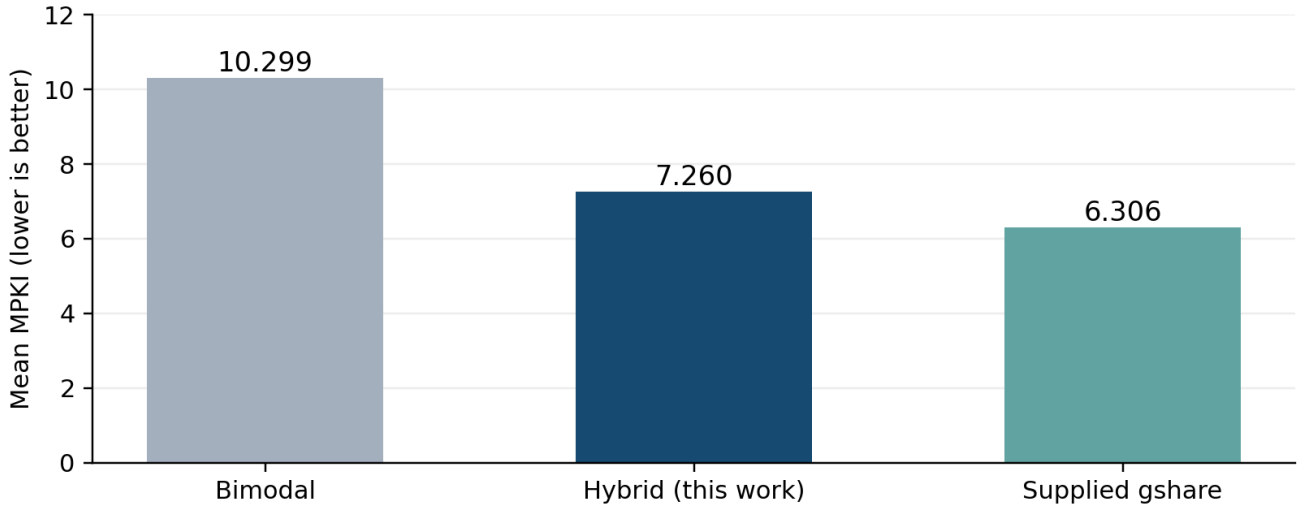


Figure 10. Mean MPKI of the three dynamic predictors. Capacities are not equal.

Comparison	Measured change for hybrid
Versus starter always taken	91.59% fewer misses per instruction
Versus bimodal-only	29.51% lower mean MPKI; 19/20 trace wins
Versus supplied gshare	15.12% higher mean MPKI; 1/20 trace wins

The hybrid is useful because the bimodal table learns a branch's general bias while global entries learn particular PC/history combinations. I see strong improvements over bimodal in workloads such as mcf (33.245 to 17.680), jess (8.809 to 1.750), and eon (9.405 to 3.139 MPKI).

Gshare performs better overall in this experiment. A plausible contributor is its larger direct-indexed table, while the hybrid must manage two-way conflicts and partial-tag aliasing. This is an interpretation, not a separately isolated causal measurement; I did not run a matched-storage study.

The twolf regression also shows that global priority is not automatically beneficial on every trace. A valid hit is allowed to override a correct bimodal prediction, exactly as the assignment specifies. The result should not be presented as a universal accuracy improvement or as a measured CPU speedup.

6 Reproduction and submission contents

Build, check, and reproduce

```
make -C src
make -C src test
make -C src sanitize
./run traces
python3 run_comparison.py
```

The no-argument Makefile target builds the required predictor. The comparison script preserves exact per-run stdout, all counts, and summaries. gzip and bzip2 must be installed at the locations configured in `src/trace.h`.

Path	Purpose
src/my_predictor.h	Required hybrid class and comparison predictors
src/predict.cc	Driver and conditional-direction metrics
tests/	Directed, reference-model, and CLI checks
traces/	20 supplied compressed traces, unchanged
baseline/	Original files used for before snapshots
results/benchmark_results.csv	All 80 measured benchmark rows
results/raw_runs.json	Commands and actual output for every benchmark run
results/test_output.txt	Captured successful build/test/sanitizer log
results/toy_trace.csv	Measured 23-event teaching example
results/code_changes.patch	Full diff of changed starter files
report/	This PDF, source snapshots, plot, and report builder
README.md	Author, dependencies, choices, limits, and submission instructions

I keep the source code, report, README, tests, and measured results together in my GitHub repository: https://github.com/ahmedshehata08-iss/branch-predictor. The original infrastructure attribution remains in place.

7 Requirement review and conclusion

Assignment requirement	Evidence / status
Implement pm_predictor member functions	Completed constructor, predict, and update; Figures 1-4.
Bimodal and global outcome prediction	Completed 4,096-counter bimodal and tagged two-way global predictor.
Global hit determines final outcome	Implemented explicitly; tested with disagreement in both directions.
Two-way option instead of four-way	Used as allowed in the assignment highlights.
PIR/loop/targets may be skipped	Excluded; no target or extra-credit completion claimed.
Build and run supplied infrastructure	Build and 80 trace runs completed; full outputs included.
Code, report, README for GitHub	Included together; repository link is on page 1 and in the README.
Before/after snapshots and author	Eight labeled snapshot pairs; author Ahmed Ahmed in code and report.

I replaced the starter behavior with a working hybrid direction predictor and verified its lookup, training, replacement, and history rules. The complete trace results show a substantial improvement over the starter and over bimodal-only prediction, with a clear tradeoff against the larger supplied gshare reference.

Limitations

The model has partial-tag aliasing, a single outstanding prediction context, no speculation recovery, and no target/loop/PIR structures. The policy choices are stated rather than presented as undisclosed Pentium M internals. MPKI alone does not measure execution time, energy, or power savings.

Sources and attribution

[1] Supplied *programming-branch-predictor.pdf*, Assignment 1, especially the scope/diagram on page 2 and Appendix B.
[2] Supplied *cbp2-infrastructure-v2*, source, traces, README, and doc/index.html; documentation credits Daniel A. Jimenez.
[3] *Sample_Tomasulo_Assignment_Report(2).pdf*, used only for report structure and before/after presentation.
No external cache implementation was used. The bundled *milenkovic_WDDD02.pdf* is titled *Demystifying Intel Branch Predictors* and is not the 2009 paper named in the assignment; the assignment's supplied diagram is the basis of this implementation.